MTGNP

Magic: The Gathering Multiplayer Network Protocol v1.0

MTGNP is a two-player, server-authoritative implementation of the Magic: The Gathering Multiplayer Network Protocol v1.0 for CSNETWK. The required base protocol and gameplay milestones are implemented. Full coverage of every card effect and graphical interfaces are intentionally left as optional work.

Base milestone status

Area	Status	Evidence
Protocol and framing	Complete	All 25 PDU types, four-byte big-endian frames, 65,535-byte limit, sender validation
Lobby and setup	Complete	Two seats, deck validation, hidden hands, random first player, London mulligan
Turn engine	Complete	Every phase/step, Draw Step, priority, Cleanup, restart
Mana, spells, and Stack	Complete	Land limits, inferred mana payment, LIFO resolution, state-based actions, more than five effects
Combat	Complete	Attackers, blockers, order, first strike, simultaneous damage, summoning sickness
Resilience	Complete	PING/PONG, priority deadlines, concession, 30-second reconnect grace, repeat games
Submission package	Locally complete	README/PDF, diagrams, AI disclosure, test matrix; LAN and cross-group checks require external participants

The automated suite currently contains 111 tests and uses only Python's standard library at runtime.

Requirements

- Python 3.11 or newer
- Two terminal windows for clients and one for the server
- TCP port 4444 reachable between the machines for a LAN game
- No third-party runtime packages

Build and test

Open PowerShell in the `MTGNP` directory:

Command / example

```
$env:PYTHONPATH = "src"
python -m unittest discover -s tests -v
```

An editable install is optional:

Command / example

```
python -m pip install -e .
```

After an editable install, `mtgnp-server` and `mtgnp-client` may be used in place of `python -m mtgnp.server` and `python -m mtgnp.client`.

Regenerate the submission PDF

The game has no third-party runtime dependency. Rebuilding the documentation PDF additionally requires ReportLab:

Command / example

```
python -m pip install reportlab
python scripts/build_readme_pdf.py
```

The stable output path is `output/pdf/README.pdf`. Regenerate it after filling in the Work Distribution Matrix or changing the README.

Run a local game

Open three PowerShell terminals in this directory and set the source path in each terminal:

Command / example

```
$env:PYTHONPATH = "src"
```

Start the authoritative server:

Command / example

```
python -m mtgnp.server --verbose
```

Start player 1:

Command / example

```
python -m mtgnp.client --player-id alice --deck decks/stack_demo_red_green.json --auto-keep --verbose
```

Start player 2:

Command / example

```
python -m mtgnp.client --player-id bob --deck decks/stack_demo_blue_black.json --auto-keep --verbose
```

`--verbose` is the required demo mode. It prints every complete PDU sent and received with direction, peer, type, sequence number, timestamp, and formatted JSON. Remove the flag for normal play. Add `--auto-pass` to both clients for an unattended protocol smoke test.

For a LAN game, run the server on one computer, allow inbound TCP port 4444 in the host firewall, determine the host's LAN IPv4 address, and give both clients that address:

Command / example

```
python -m mtgnp.client --host 192.168.1.10 --player-id alice --deck decks/stack_demo_red_green.json --auto-keep --verbose
```

Terminal controls

At a priority prompt the client accepts:

Command / example

```
pass
concede
land CARD_ID
cast CARD_ID [TARGET ...]
activate SOURCE_ID ABILITY_INDEX [TARGET ...]
```

Examples:

Command / example

```
land mountain_001
cast lightning_bolt_001 bob
cast counterspell_001 stk_0001
cast doom_blade_001 goblin_guide_001
activate prodigal_sorcerer_001 0 bob
activate millstone_001 0 bob
```

Mana payment is inferred from the catalog and paid by atomically tapping legal sources. Diagnostic overrides are available:

Command / example

```
cast doom_blade_001 goblin_guide_001 --mana B=1,X=1
activate millstone_001 0 bob --mana X=2
```

Combat prompts accept attacker IDs separated by spaces, none, and blocker pairs such as `phantasmal_bear_001=goblin_guide_001`. The attacker is prompted for damage order when several creatures block one attacker.

Rules behavior that may not be obvious

- The first player skips only the Draw Step of turn 1. The second player draws on turn 2, and the first player begins drawing on turn 3.
- Playing a land is not casting a spell and does not require mana. Only the active player may play one land during either of that player's Main Phases.
- Receiving priority during the opponent's turn permits responses such as Instants, but it does not make the responding player the active player.
- Lands remain tapped until their controller's next Untap Step. The terminal's `Available mana` line counts currently untapped mana sources.
- Both players must pass consecutively to resolve the top Stack item or advance an empty priority window.

Disconnect and reconnect

An unexpected in-game connection loss pauses the authoritative game and reserves that seat for 30 seconds. The server cancels the outstanding priority deadline while paused.

To reconnect, rerun the same client command before the grace period ends. The `PLAYER_READY` PDU acts as the reconnect handshake and must contain the exact same player ID and ordered deck list. The server then:

1. verifies the reserved identity and deck;
2. sends fresh personalized authoritative state;
3. issues a fresh priority, combat, trigger, mulligan, or discard token; and
4. continues the original game without reshuffling or resetting it.

A changed identity or deck is rejected and the reservation remains active. If the timer expires, the connected opponent receives `GAME_OVER` with reason `DISCONNECT`. The grace duration can be configured for demonstrations:

Command / example

```
python -m mtgnp.server --reconnect-grace 30 --verbose
```

Architecture

Diagram source

```
flowchart LR
    C1["Terminal client: player 1"] <-->|"Framed JSON over TCP"| S["Authoritative MTGNP server"]
    C2["Terminal client: player 2"] <-->|"Framed JSON over TCP"| S
    S --> P["Protocol validation and tracing"]
    S --> L["Two-seat lobby and lifecycle"]
    S --> G["Game, Stack, abilities, triggers, combat"]
    G --> D["Fixed CSV card catalog"]
```

The clients send intent only. They never determine whether an action succeeds or calculate a winner. Every accepted mutation occurs in the server game engine, and each `GAME_STATE_UPDATE` replaces the client's local view.

The game lifecycle is:

Diagram source

```
flowchart LR
    L["LOBBY"] --> S["GAME_SETUP"] --> M["MULLIGAN"] --> G["IN_GAME"] --> O["GAME_OVER"] --> L
    G -. "connection lost" .-> R["RECONNECT GRACE"]
    R -->|"same player and deck"| G
    R -->|"30 seconds expires"| O
```

Implemented effects

The base rubric requires at least five effects. The implementation exceeds that gate with spell effects, activated abilities, and triggered abilities.

- Spells: Lightning Bolt, Unsummon, Counterspell, Giant Growth, Doom Blade

- Activated abilities: Prodigal Sorcerer, Royal Assassin, Millstone, Rod of Ruin
- Triggered abilities: Goblin Guide, Monastery Swiftspear, Phantasmal Bear, Gray Merchant of Asphodel, Gravedigger

Dedicated demonstrations are available in `decks/ability_demo_*.json` and `decks/trigger_demo_*.json`.

Test coverage

Automated tests cover catalog reconciliation, all PDU shapes, malformed and fragmented framing, hidden information, mulligans, turn transitions, draws, land and mana rules, Stack ordering and countering, state-based actions, combat, activated and triggered abilities, timeouts, heartbeat behavior, concession, same-connection restart, reconnect success, reconnect rejection, and reconnect expiry.

The external checks in `docs/INTEROPERABILITY_TEST_MATRIX.md` must be completed on the actual demo machines. They are deliberately not marked as passed in advance.

Known limitations and RFC interpretations

- MTGNP defines reconnect as required but defines no reconnect/session PDU. This implementation reuses `PLAYER_READY` with the same player ID and exact ordered deck. This is an implementation-defined extension using an existing PDU rather than adding a non-standard message type.
- Player IDs are not authenticated because MTGNP 1.0 defines no authentication. The reconnect check prevents accidental seat takeover, not a malicious peer that already knows the original player ID and deck.
- A legal deck may contain fewer than seven cards. Setup draws all cards that are available; a later required draw from an empty library loses the game.
- Each physical card instance ID may occur in only one submitted deck.
- Each physical server-to-client PDU receives a distinct monotonically increasing server sequence number, including separate broadcast copies. The explicit Section 11 exception reissues the current token unchanged after a rejected action while that player still holds priority.
- Five spell effects and selected activated/triggered abilities are supported. Full behavior for every catalog card is optional bonus work and is not yet implemented.
- The required client is terminal-based. A graphical interface is optional.

Work Distribution Matrix

Replace the member headings and every `Record actual contribution` cell before submission. Do not claim work that a member did not perform.

Task / Feature	Member 1	Member 2	Member 3	Member 4
TCP server, connections, framing, dispatch	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
Lobby, setup, and mulligan	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
Turn and phase engine	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
Priority, Stack, spells, and abilities	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
Combat system	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
Client and state rendering	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
All 25 PDU types	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
Errors, heartbeat, timeout, and reconnect	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
Verbose PDU tracing	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
Tests and interoperability	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution
README, diagrams, and disclosure	Record actual contribution	Record actual contribution	Record actual contribution	Record actual contribution

AI Usage

OpenAI Codex was used as a learning and development assistant to analyze the provided MTGNP specification and CSV catalog, propose milestone boundaries, explain MTG rules and protocol behavior, draft and revise Python implementation code, create automated tests, improve terminal readability, diagnose gameplay issues, and prepare documentation. Its output was applied to the local project through an iterative workflow and checked with the automated test suite.

Before submission, every group member must personally review the implementation, run the tests, perform the manual interoperability checks, and be able to explain the generated or assisted code. Update this disclosure if any other AI tool is used.

Project documentation

- `docs/FOUNDATION_DECISIONS.md` - catalog and RFC interpretation decisions
- `docs/TRANSPORT_DESIGN.md` - TCP framing, connection wrapper, and tracing
- `docs/LOBBY_DESIGN.md` - lobby and ready validation
- `docs/GAME_SETUP_DESIGN.md` - setup, hidden state, and mulligans
- `docs/TURN_ENGINE_DESIGN.md` - turns, priority, draws, and cleanup
- `docs/STACK_AND_SPELLS_DESIGN.md` - lands, mana, spells, Stack, and effects
- `docs/COMBAT_DESIGN.md` - combat validation and damage
- `docs/ACTIVATED_ABILITIES_DESIGN.md` - activated-ability registry and flow
- `docs/TRIGGERED_ABILITIES_DESIGN.md` - triggers, choices, and APNAP ordering
- `docs/RESILIENCE_DESIGN.md` - timeout, heartbeat, concession, and reconnect
- `docs/ARCHITECTURE_AND_DEMO.md` - component and sequence walkthrough
- `docs/INTEROPERABILITY_TEST_MATRIX.md` - required external test record
- `docs/SUBMISSION_CHECKLIST.md` - final packaging and demo checklist